

What is Backend?

Backend = **Everything the user does NOT see** but depends on.

Frontend	Backend
Buttons, UI, forms	Data, users, login, business logic
HTML, CSS, JS	Node, DB, APIs
Browser	Server

Backend is responsible for:

- ✓ User accounts
- ✓ Saving data
- ✓ Authentication
- ✓ Security
- ✓ Payments
- ✓ API communication
- ✓ Scaling

Frontend without backend = **static website**

Backend + frontend = **real product**

What is a Server?

A **server** is simply a **computer that runs 24/7** waiting for requests.

When user opens a site:

Browser → Server → Response

Server jobs:

- Accept request
- Process logic
- Talk to database
- Send response

Node.js lets **JavaScript become a server language**

3 What is Node.js?

Node.js = **JavaScript runtime for backend**

Before Node:

JS only ran in browser

After Node:

JS can:

- Read files
- Talk to databases
- Handle users
- Run servers

Angela uses Node so you can use **one language (JS) everywhere**

4 What is Express.js?

Express = **backend web framework for Node**

Without Express:

You write 200+ lines to make a server

With Express:

5 lines → server ready

Express handles:

- Routing
- Requests
- Responses
- Middleware
- APIs

Example:

```
app.get("/", (req, res) => {  
  res.send("Hello")  
})
```

This means:

“If user goes to /, send Hello”

HTTP Requests

Frontend talks to backend using **HTTP**

Main types:

Method	Meaning
--------	---------

GET	Get data
-----	----------

POST Send
data

PUT Update

DELETE Delete

Example:

- Login form → POST
- Profile → GET
- Edit user → PUT
- Delete account → DELETE

npm & Packages

npm = **Node Package Manager**

It installs:

- Express
- Mongoose
- Body parser
- Authentication libraries

This is how backend developers build fast.

What is a Database?

A database stores:

- Users
- Passwords
- Orders
- Messages
- Anything permanent

Angela uses **MongoDB**

MongoDB is:

- NoSQL
- Document-based
- JSON-like

Why MongoDB?

- Works naturally with JavaScript
- Flexible schema
- Easy for startups

Mongoose

Mongoose = MongoDB toolkit for Node

It gives:

- Schema
- Validation
- Models

Schema = blueprint of data

Example:

```
User {  
    email: String  
    password: String  
}
```

Model = actual object you save.

Authentication (Login System)

Angela teaches:

User signs up →
Password hashed →
Saved in DB →
User logs in →
Password verified →
Session created

Never store raw passwords ❌
Always hash using bcrypt ✔

Sessions & Cookies

HTTP is stateless — it forgets users.

So backend uses:

- Cookies
- Sessions

Flow:

Login → server creates session → browser stores cookie → every request sends cookie → backend knows who you are

This is how:

- Gmail
 - Instagram
 - Amazon stay logged in
-

1 1 Security

Important backend security:

- Hash passwords
- Use HTTPS
- Validate input
- Prevent SQL injection
- Protect routes

Backend = **guardian of your app**

1 2 APIs

API = backend endpoint that returns data

Example:

`/api/users`

`/api/orders`

`/api/products`

Frontend hits APIs → backend sends JSON → frontend shows UI

This is how:

- Mobile apps
 - Websites
 - AI models
talk to the same backend
-

REST Architecture

REST = standard for APIs

Rules:

- Use nouns
- Use HTTP verbs

Example:

GET /users

POST /users

PUT /users/12

DELETE /users/12

This is **industry standard**

Deployment

Localhost → Real world

Angela shows:

- Hosting backend
- Connecting database
- Making app public